JSPM's
Rajarshi Shahu College of Engineering,
Tathawade, Pune – 33.
Department of Computer Science and Business Systems
AY: 2026-27 Sem: II

Mini Project Report On

"Customer Relationship Management (CRM) System"

Submitted by:

Harish Dandekar (PRN: RBT25CB006)

Sanskar Gaikwad (PRN: RBT25CB009)

Under the guidance of

Prof. Rutuja S. Bachhav

Course: DBMS

Academic Year: **2026–27 (Semester III)**

Table of Contents

1. Abstract 3
2. Introduction 6
3. Literature Survey / Existing System 6
4. Proposed System 7
5. Methodology / Implementation 8
6. Results and Discussion 10
7. Conclusion and Future Scope 19
8. References 20
9. Appendix (Optional) 20

1. Course Outcomes:

CO1: Design a database schema and 2 –tier application for a given problem statement.

CO2: Design SQL DDL/DML quries to create, secure, populate, maintain, and query a database using integrity constraint, stored function/procedure, cursor, trigger using PL/SQL block.

CO2: Design NO SQL query using mongoDB platform.

2. PO Mapped

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing,
engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems
PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modeling, analysis & interpretation of data to provide valid conclusions.
PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modeling recognizing their limitations to solve complex engineering problems.
PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to

one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

3. PSO Mapped

PSO 1: Integrated Technical Proficiency: Ability to analyze, designs, and implement comprehensive software and business systems using modern tools and techniques, ensuring intelligent, efficient, and sustainable solutions.

PSO 2: Data-Driven Decision Making: Ability to apply mathematical and data analytics methods including algorithms, statisticalmodels, and data structures to solve complex computational and business problems and support innovation.

PSO 3: Interdisciplinary Collaboration & Professionalism: Ability to effectively collaborate in multidisciplinary teams, integrating technical and business perspectives, and to communicate and manage projects with professional ethics and societal responsibility.

4. SDG Goals Mapped

SDG 4: Quality Education: Develops students' practical skills in database design, SQL, PL/SQL, NoSQL, and modern database technologies.

SDG 8: Decent Work and Economic Growth: Enhances students' technical and problem-solving skills required for employment and professional development in the IT industry.

SDG 9: Industry, Innovation and Infrastructure: Promotes the development of technology-based database solutions using SQL, PL/SQL, and MongoDB.

SDG 12: Responsible Consumption and Production: Encourages efficient data management, storage, and utilization of computing resources.

5. Abstract

The Customer Relationship Management (CRM) System is a database-driven application designed to organize customer information and support the management of leads, products, and sales deals. The project demonstrates how a relational database can be connected to a web-based interface through a backend API. MySQL is used to store structured records and maintain relationships between entities, while HTML, CSS, and JavaScript provide the user interface. Node.js with Express handles HTTP requests and communicates with MySQL using the mysql2 package. The application supports viewing customer records through API endpoints and can be extended to perform create, update, and delete operations. Primary keys and foreign-key constraints help preserve data integrity; for example, a customer referenced by a deal cannot be deleted until dependent records are handled. The project follows a layered architecture that separates presentation, application logic, and data storage. It provides practical experience with relational schema design, SQL queries, API development, and database constraint handling. The implementation is intended as an academic prototype and can be expanded with authentication, reporting, validation, and role-based access control.

6. Introduction

- Background / Motivation
  Organizations need a reliable way to keep customer details, sales leads, products, and deals organized. Managing this information in separate files can lead to duplication, inconsistent records, and difficulty tracking sales activity. A CRM database provides a central place to store related information and retrieve it efficiently.
- Problem Definition
  The project addresses the need for a simple web-based system that stores and retrieves CRM records using a relational database. The system must connect a browser interface to MySQL safely through a backend rather than allowing browser code to connect directly to the database.
- Objectives
  • Design a relational schema for customers, employees, leads, products, and deals.
  • Apply primary keys, foreign keys, and suitable data types to maintain integrity.
  • Build a web interface using HTML, CSS, and JavaScript.
  • Create REST-style API endpoints with Node.js and Express.
  • Execute SQL operations through the backend and demonstrate linked-record constraints.

7. Literature Survey / Existing System

- Review of related tools or existing systems
  Traditional spreadsheets are easy to start with but provide limited relationship enforcement and can become difficult to maintain as records grow. Commercial CRM platforms offer advanced automation, dashboards, and access controls, but may be excessive for a small academic demonstration. A custom database application is suitable for demonstrating the core principles of data modelling, SQL, and client-server communication.
- Gaps identified
  A basic manual system may allow duplicate customer data, inconsistent updates, and weak traceability between leads and deals. The proposed academic prototype centralizes records and uses relational constraints. It does not yet provide production-level authentication, audit logs, automated backups, or advanced analytics.

8. Proposed System

- System Overview

  The CRM System consists of a browser-based frontend, an Express backend, and a MySQL database. The frontend sends HTTP requests to API routes. The backend validates the request, executes parameterized SQL queries, and returns JSON responses. MySQL stores the records and enforces the defined relationships.

- Features / Scope

  • View customer records through the /api/customers endpoint.
  • Organize CRM entities such as customers, employees, leads, products, and deals.
  • Support CRUD operations where corresponding routes and interface controls are implemented.
  • Enforce primary-key and foreign-key relationships.
  • Display operation success or error messages to the user.

  Scope limitation: this version is a local academic prototype and does not claim enterprise security or deployment readiness.

- System Architecture Diagram

  [Browser: HTML / CSS / JavaScript]
  ▼ HTTP request / JSON response
  [Node.js + Express API (server.js)]
  ▼ SQL queries via mysql2
  [MySQL Database: miniproject]
  Tables: Customers, Employees, Leads, Products, Deals

  The browser communicates only with the API. Database credentials remain in the backend configuration, not in client-side JavaScript.

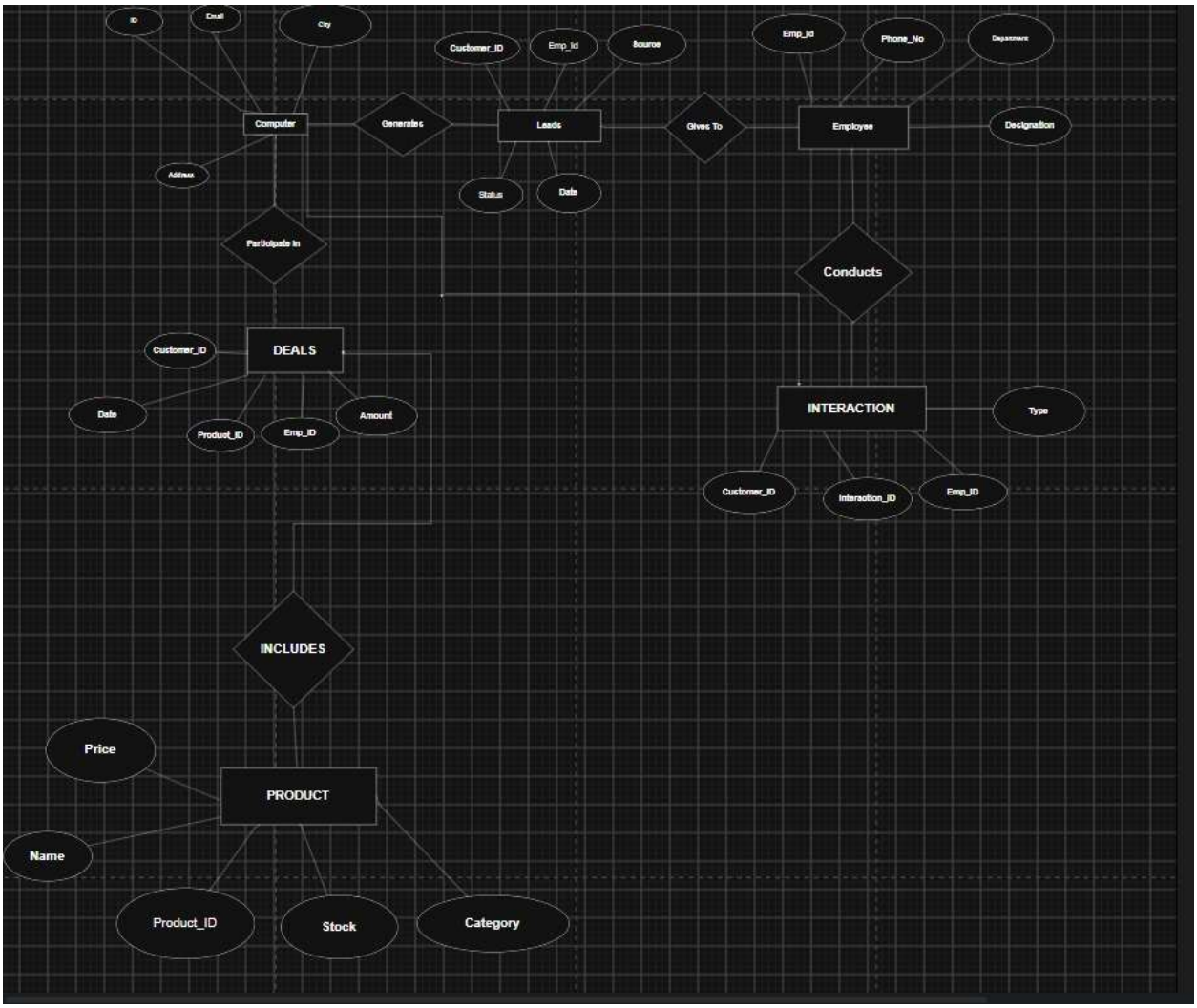
9. Methodology / Implementation

- Tools and Technologies Used
  • Frontend: HTML, CSS, JavaScript
  • Backend: Node.js and Express.js
  • Database: MySQL
  • Database driver: mysql2
  • Development environment: Visual Studio Code and a local browser
  • API testing: browser access to local endpoints such as http://localhost:3000/api/customers
- Modules Description
  1. Customer module: stores and retrieves customer details.
  2. Lead module: records lead source, status, date, and related customer/employee identifiers.
  3. Product module: stores product information used by sales deals.
  4. Deal module: records sales opportunities and their customer/product relationships.
  5. API module: exposes HTTP routes and returns JSON results.
  6. Database integrity module: uses keys and constraints to prevent invalid relationships.
- Algorithms / Frameworks
  1. The user performs an action in the browser.
  2. JavaScript sends an HTTP request to an Express route.
  3. Express reads and validates request parameters/body.
  4. The backend executes an SQL statement using the MySQL connection.
  5. MySQL returns rows or an operation result.
  6. Express sends a JSON response and an appropriate HTTP status.
  7. The frontend updates the page or displays an error message.

  Foreign-key example: deleting a customer referenced by a deal is rejected by MySQL unless dependent records are first reassigned or deleted according to the intended business rule.
- Screenshots or Outputs
  Insert actual screenshots from your running project here before submission.
  Recommended evidence:
  • CRM homepage showing navigation/modules.
  • Customer list in the browser.
  • Successful response from /api/customers.
  • A form submitting a record successfully.
  • Validation or foreign-key error message for a blocked delete.

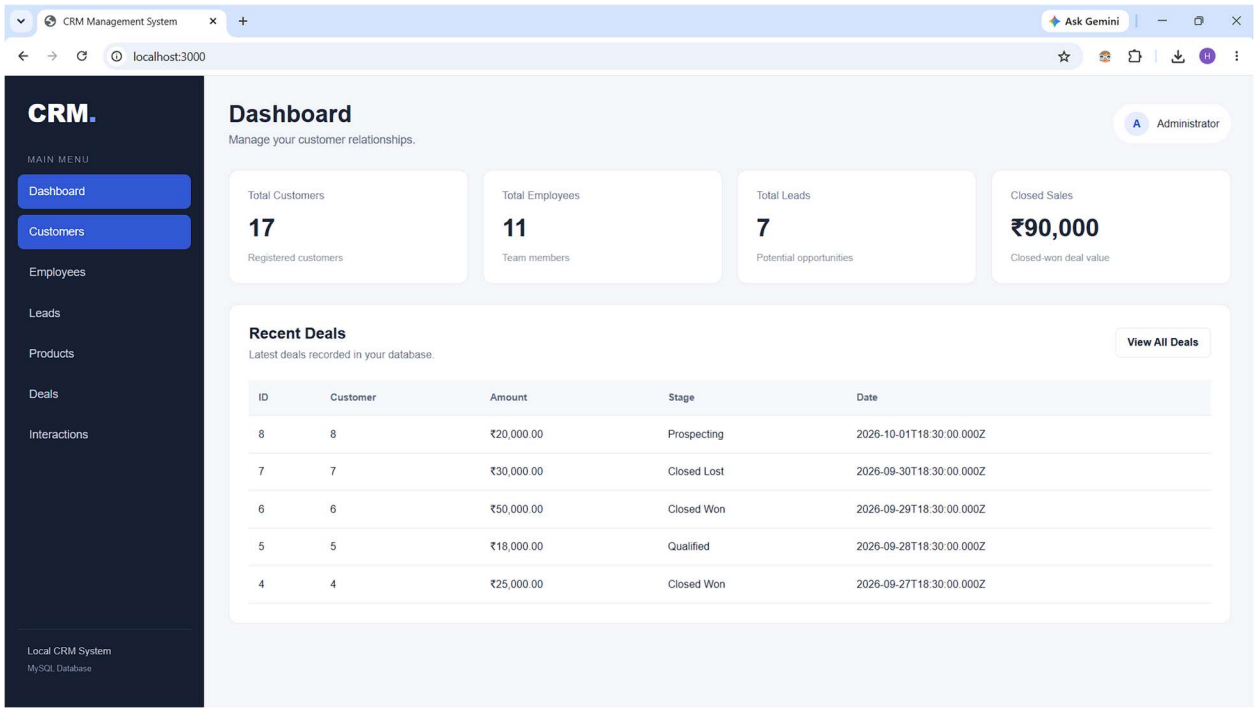
  Screenshots were not included in the supplied report template, so these are marked for insertion rather than fabricated.

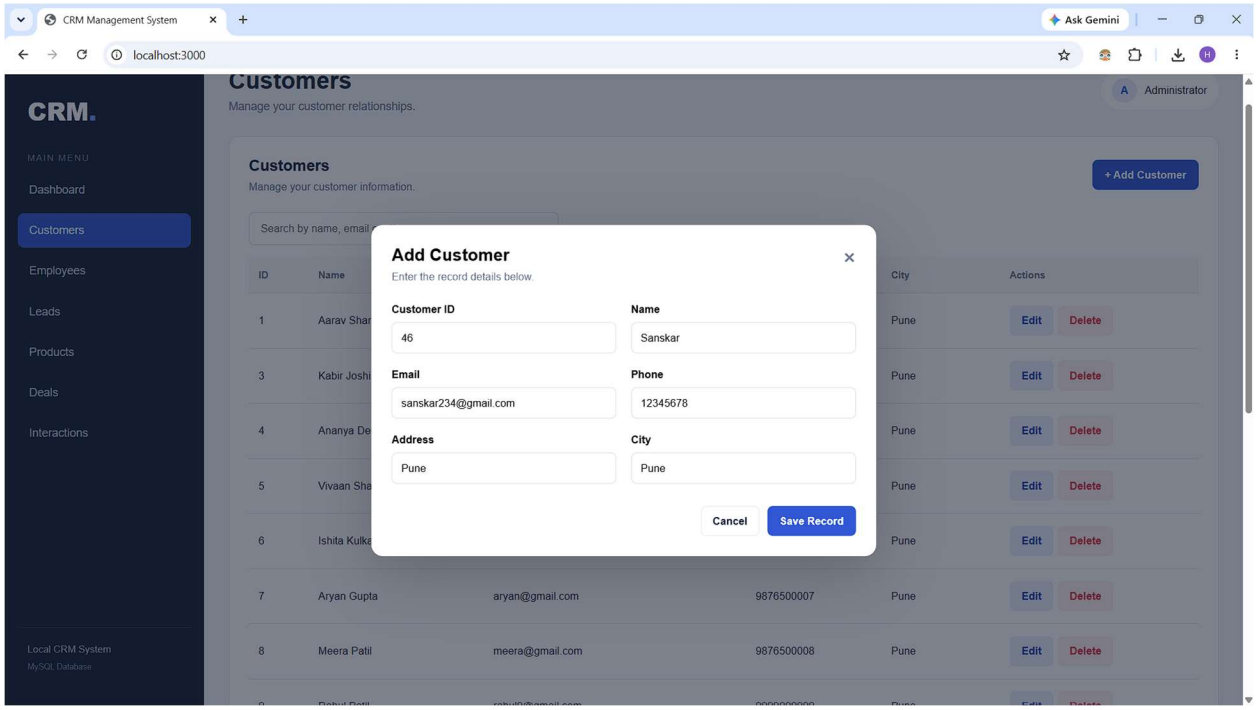
EER Diagram:

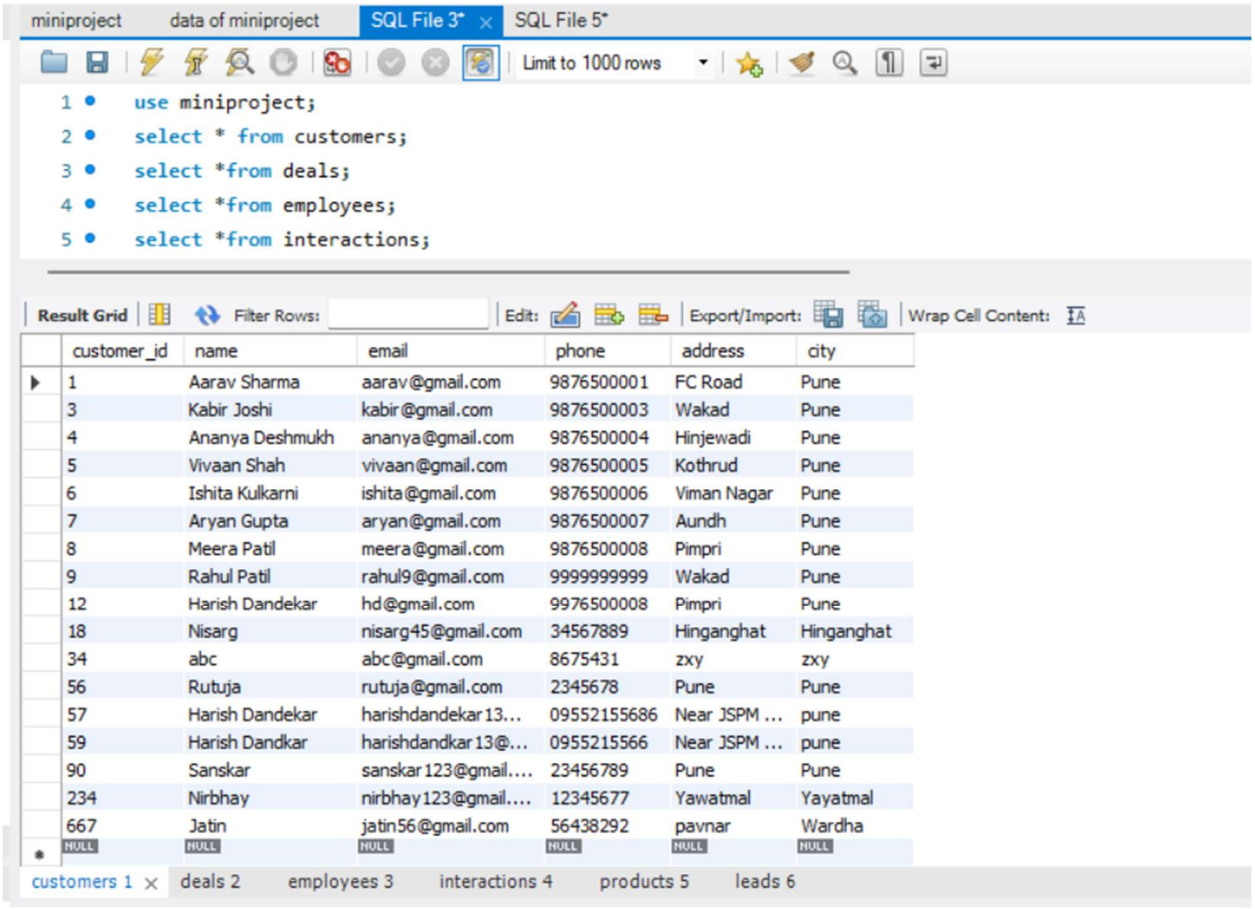

ID
Email
City
Customer_ID
Emp_Id
Source
Emp_Id
Phone_No
Department
Computer
Generates
Leads
Gives To
Employee
Designation
Address
Status
Date
Participate In
Conducts
Customer_ID
DEALS
Date
Product_ID
Emp_ID
Amount
INTERACTION
Type
Customer_ID
Interaction_ID
Emp_ID
INCLUDES
Price
PRODUCT
Name
Product_ID
Stock
Category

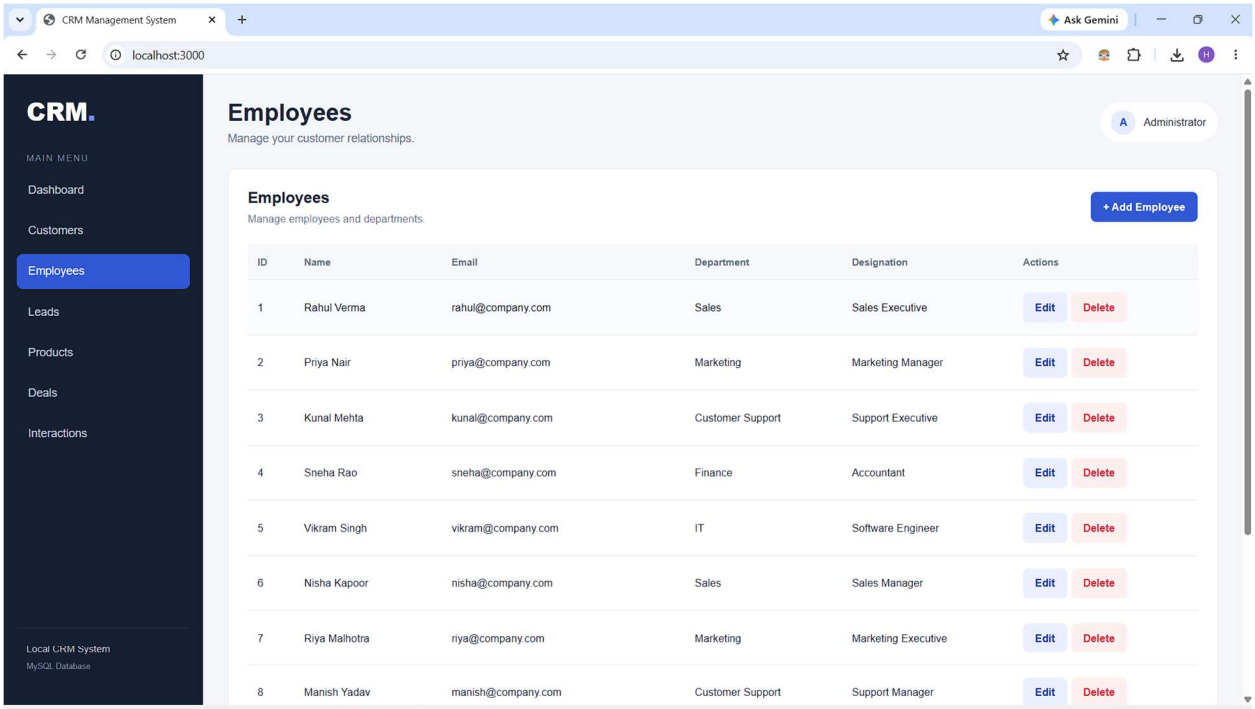
Results and Discussion

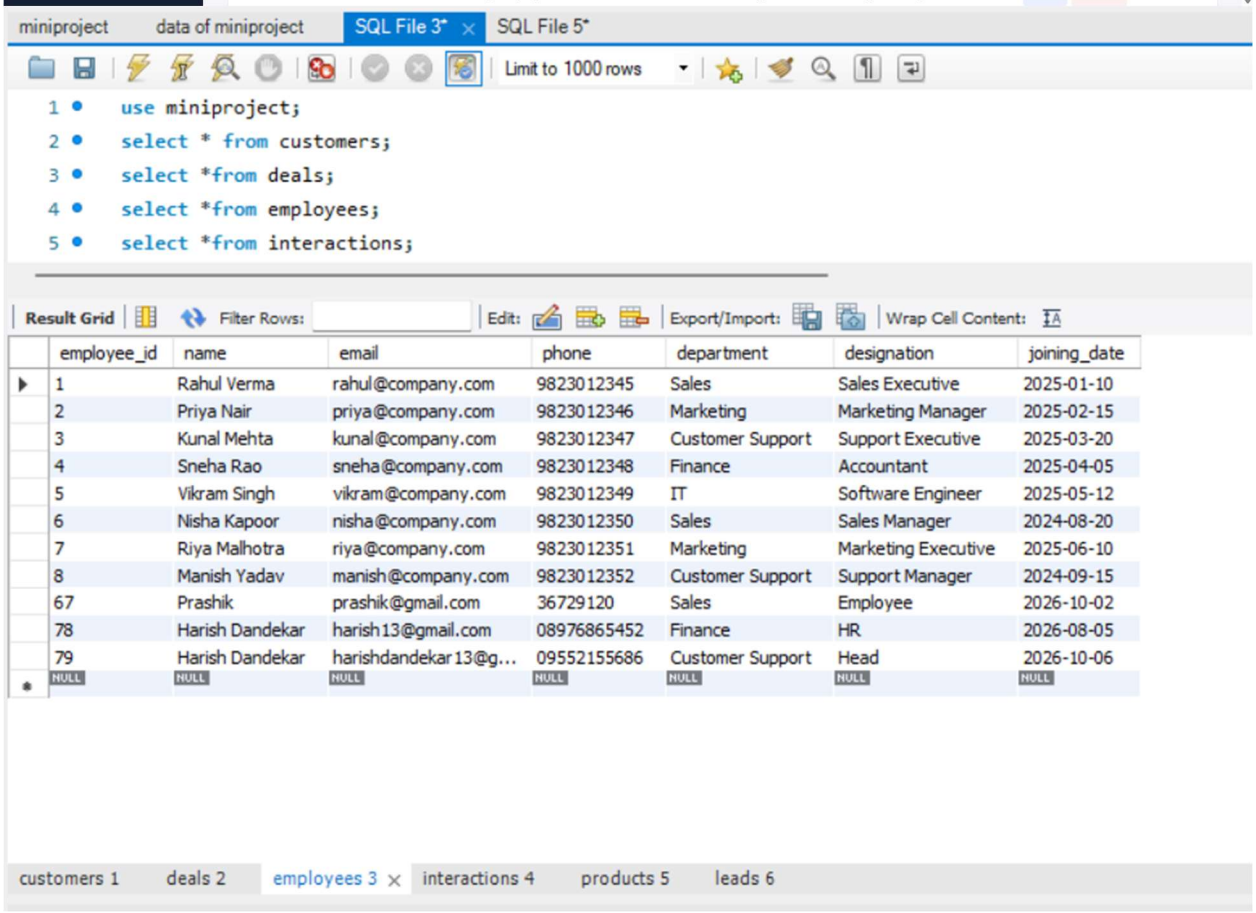
OUTPUT:-



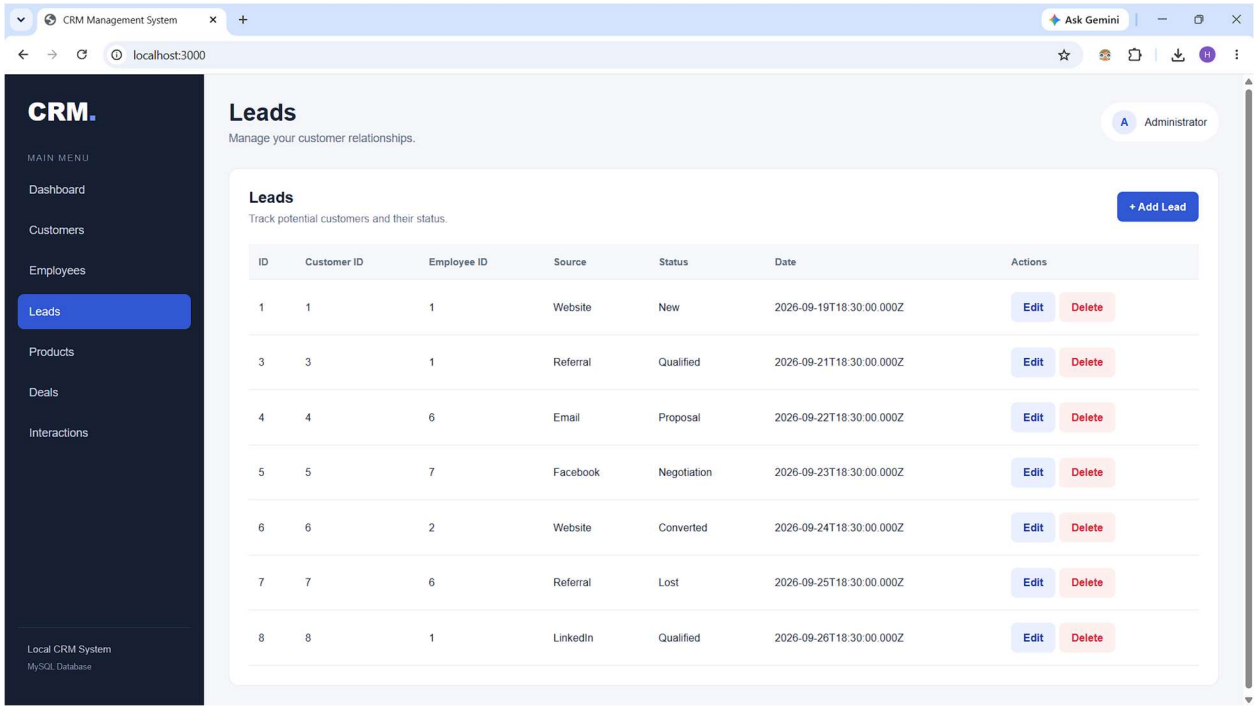

CRM Management System
Ask Gemini
localhost:3000
CRM.
MAIN MENU
Dashboard
Customers
Employees
Leads
Products
Deals
Interactions
Local CRM System
MySQL Database
Customers
Manage your customer relationships.
Administrator
Customers
Manage your customer information.
+ Add Customer
Search by name, email
ID
Name
City
Actions
1
Aarav Shar
Pune
Edit
Delete
3
Kabir Joshi
Pune
Edit
Delete
4
Ananya De
Pune
Edit
Delete
5
Vivaan Sha
Pune
Edit
Delete
6
Ishita Kulka
Pune
Edit
Delete
7
Aryan Gupta
aryan@gmail.com
9876500007
Pune
Edit
Delete
8
Meera Patil
meera@gmail.com
9876500008
Pune
Edit
Delete
Add Customer
Enter the record details below.
Customer ID
46
Name
Sanskar
Email
sanskar234@gmail.com
Phone
12345678
Address
Pune
City
Pune
Cancel
Save Record

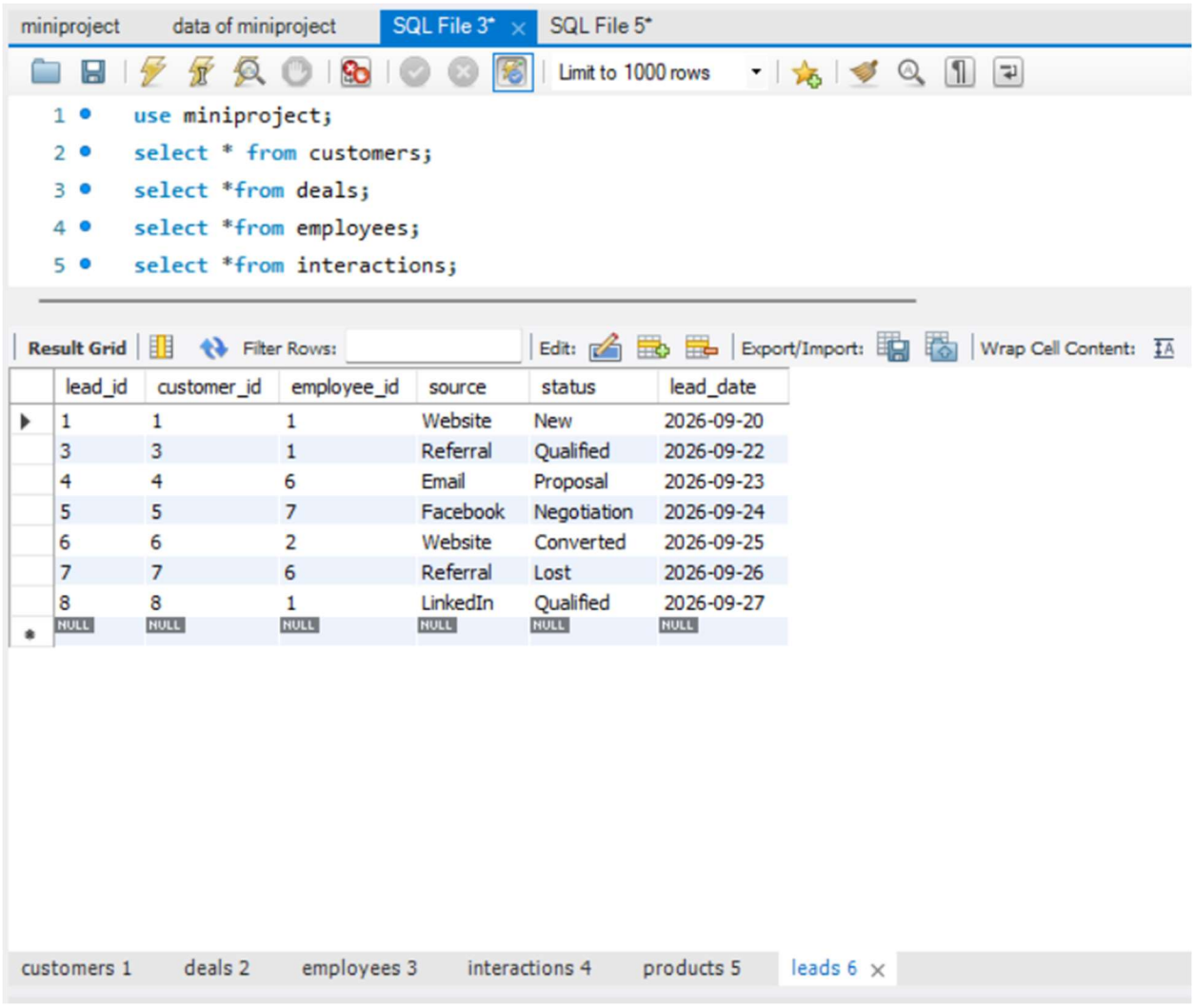

miniproject
data of miniproject
SQL File 3*
SQL File 5*
Limit to 1000 rows
use miniproject;
select * from customers;
select *from deals;
select *from employees;
select *from interactions;
Result Grid
Filter Rows:
Edit:
Export/Import:
Wrap Cell Content:
customer_id | name | email | phone | address | city
1 | Aarav Sharma | aarav@gmail.com | 9876500001 | FC Road | Pune
3 | Kabir Joshi | kabir@gmail.com | 9876500003 | Wakad | Pune
4 | Ananya Deshmukh | ananya@gmail.com | 9876500004 | Hinjewadi | Pune
5 | Vivaan Shah | vivaan@gmail.com | 9876500005 | Kothrud | Pune
6 | Ishita Kulkarni | ishita@gmail.com | 9876500006 | Viman Nagar | Pune
7 | Aryan Gupta | aryan@gmail.com | 9876500007 | Aundh | Pune
8 | Meera Patil | meera@gmail.com | 9876500008 | Pimpri | Pune
9 | Rahul Patil | rahul9@gmail.com | 9999999999 | Wakad | Pune
12 | Harish Dandekar | hd@gmail.com | 9976500008 | Pimpri | Pune
18 | Nisarg | nisarg45@gmail.com | 34567889 | Hinganghat | Hinganghat
34 | abc | abc@gmail.com | 8675431 | zxy | zxy
56 | Rutuja | rutuja@gmail.com | 2345678 | Pune | Pune
57 | Harish Dandekar | harishdandekar13... | 09552155686 | Near JSPM ... | pune
59 | Harish Dandkar | harishdandkar13@... | 0955215566 | Near JSPM ... | pune
90 | Sanskar | sanskar123@gmail.... | 23456789 | Pune | Pune
234 | Nirbhay | nirbhay123@gmail.... | 12345677 | Yawatmal | Yayatmal
667 | Jatin | jatin56@gmail.com | 56438292 | pavnar | Wardha
customers 1
deals 2
employees 3
interactions 4
products 5
leads 6



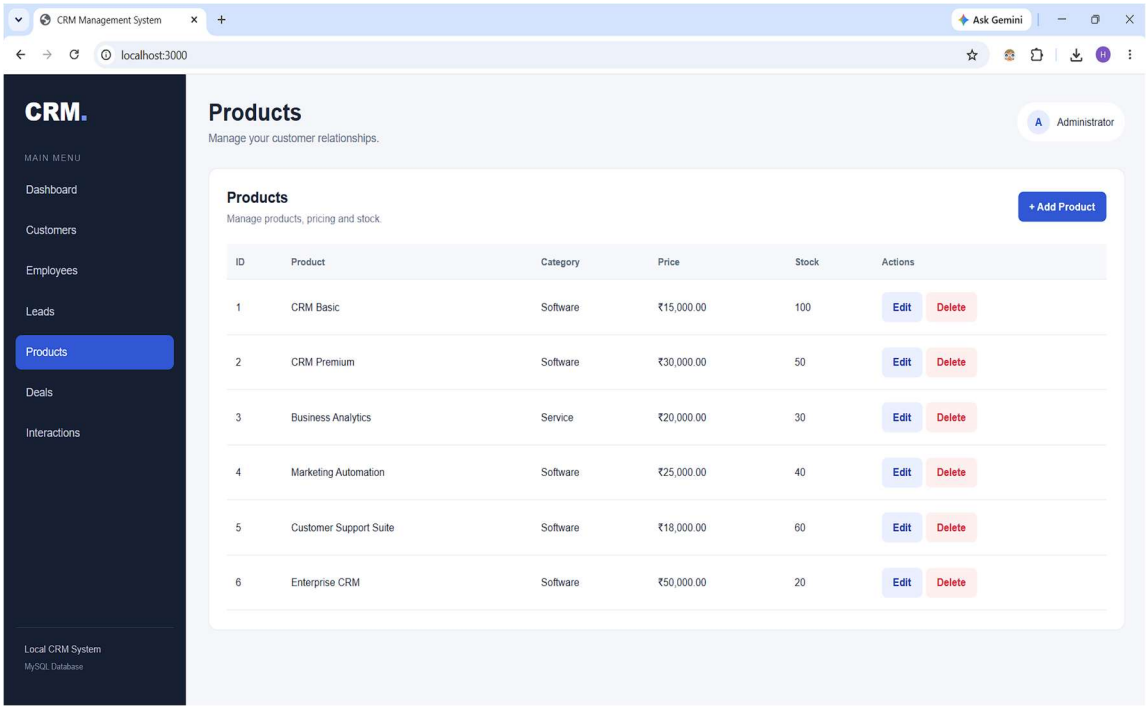


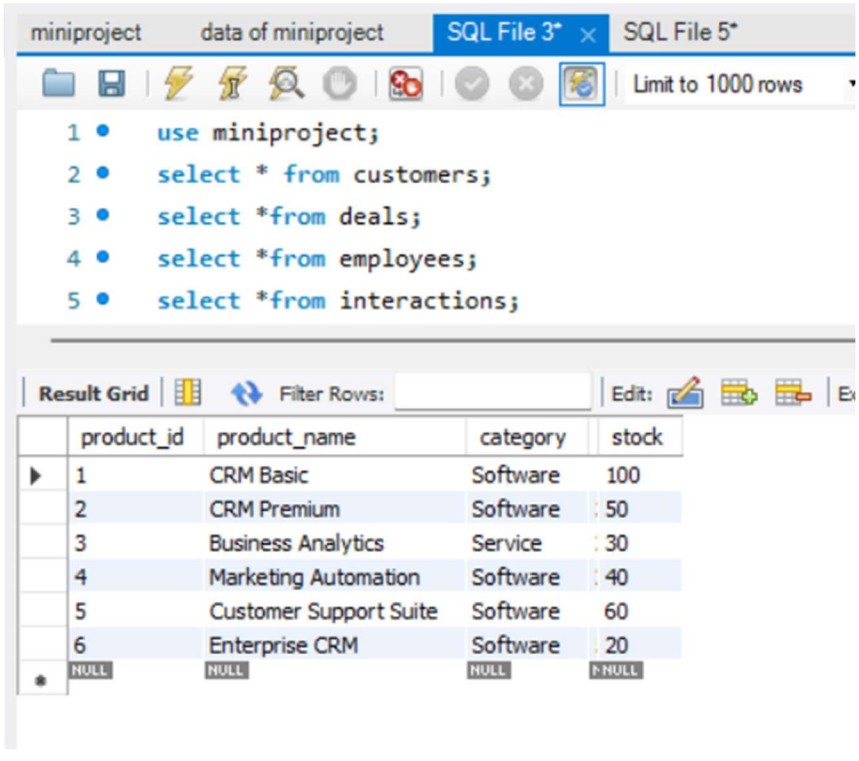
employee_id	name	email	phone	department	designation	joining_date
1	Rahul Verma	rahul@company.com	9823012345	Sales	Sales Executive	2025-01-10
2	Priya Nair	priya@company.com	9823012346	Marketing	Marketing Manager	2025-02-15
3	Kunal Mehta	kunal@company.com	9823012347	Customer Support	Support Executive	2025-03-20
4	Sneha Rao	sneha@company.com	9823012348	Finance	Accountant	2025-04-05
5	Vikram Singh	vikram@company.com	9823012349	IT	Software Engineer	2025-05-12
6	Nisha Kapoor	nisha@company.com	9823012350	Sales	Sales Manager	2024-08-20
7	Riya Malhotra	riya@company.com	9823012351	Marketing	Marketing Executive	2025-06-10
8	Manish Yadav	manish@company.com	9823012352	Customer Support	Support Manager	2024-09-15
67	Prashik	prashik@gmail.com	36729120	Sales	Employee	2026-10-02
78	Harish Dandekar	harish13@gmail.com	08976865452	Finance	HR	2026-08-05
79	Harish Dandekar	harishdandekar13@g...	09552155686	Customer Support	Head	2026-10-06
NULL	NULL	NULL	NULL	NULL	NULL	NULL

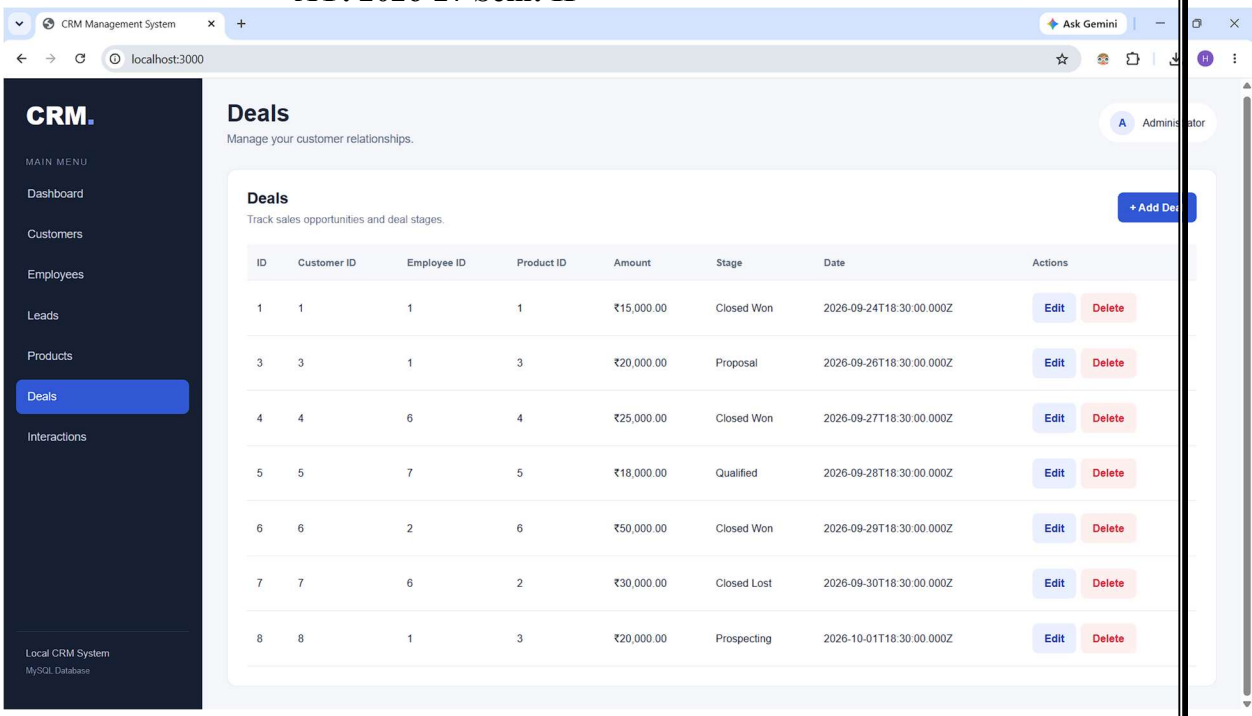

CRM Management System
Ask Gemini
localhost:3000
CRM.
MAIN MENU
Dashboard
Customers
Employees
Leads
Products
Deals
Interactions
Local CRM System
MySQL Database
Leads
Manage your customer relationships.
A
Administrator
Leads
Track potential customers and their status.
+ Add Lead
ID
Customer ID
Employee ID
Source
Status
Date
Actions
1 1 1 Website New 2026-09-19T18:30:00.000Z Edit Delete
3 3 1 Referral Qualified 2026-09-21T18:30:00.000Z Edit Delete
4 4 6 Email Proposal 2026-09-22T18:30:00.000Z Edit Delete
5 5 7 Facebook Negotiation 2026-09-23T18:30:00.000Z Edit Delete
6 6 2 Website Converted 2026-09-24T18:30:00.000Z Edit Delete
7 7 6 Referral Lost 2026-09-25T18:30:00.000Z Edit Delete
8 8 1 LinkedIn Qualified 2026-09-26T18:30:00.000Z Edit Delete

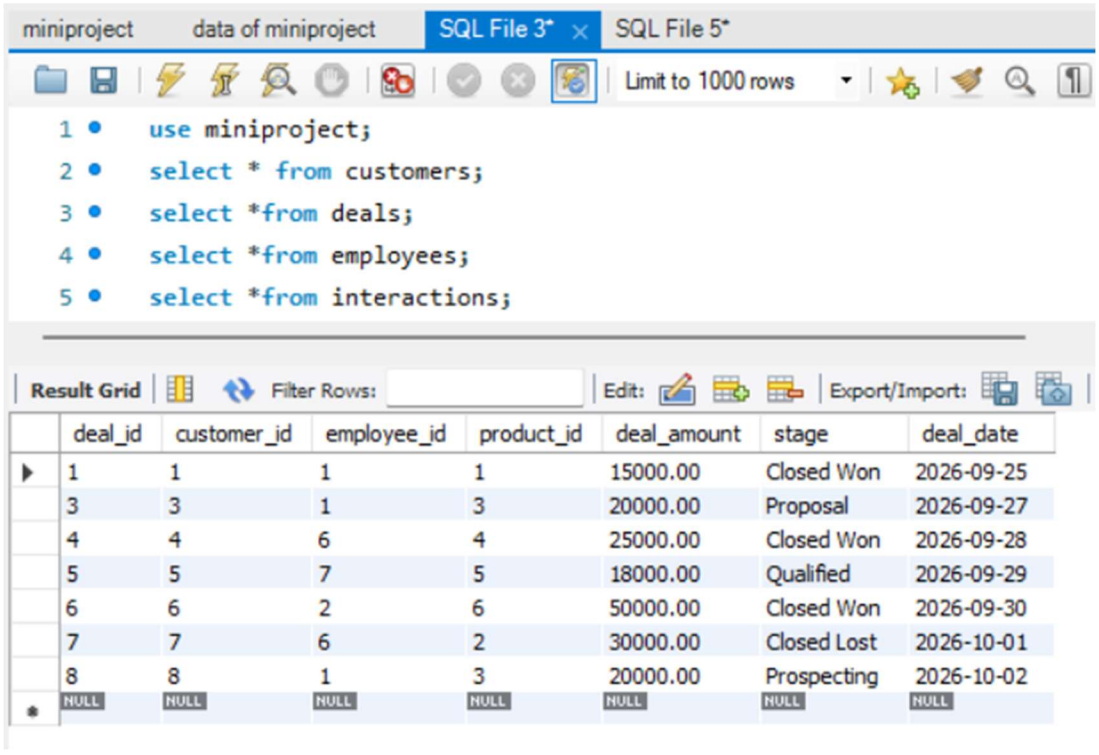


lead_id	customer_id	employee_id	source	status	lead_date
1	1	1	Website	New	2026-09-20
3	3	1	Referral	Qualified	2026-09-22
4	4	6	Email	Proposal	2026-09-23
5	5	7	Facebook	Negotiation	2026-09-24
6	6	2	Website	Converted	2026-09-25
7	7	6	Referral	Lost	2026-09-26
8	8	1	LinkedIn	Qualified	2026-09-27
NULL	NULL	NULL	NULL	NULL	NULL

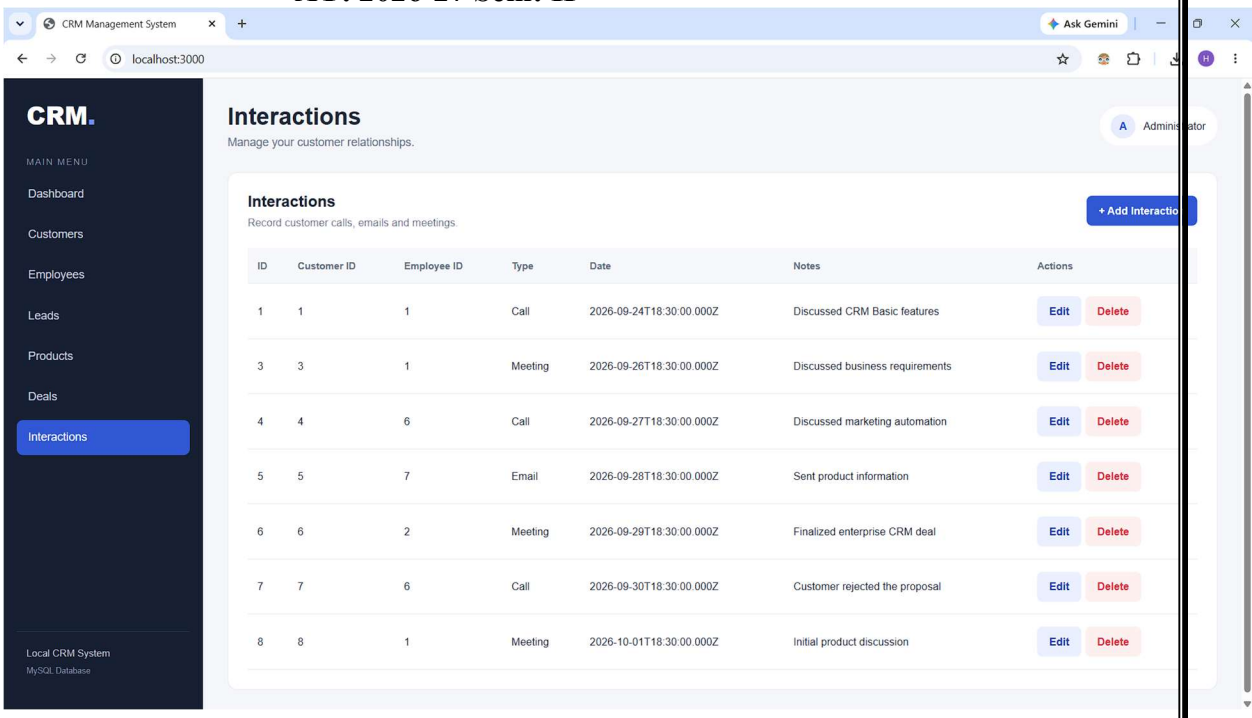

CRM Management System
Ask Gemini
localhost:3000
CRM.
MAIN MENU
Dashboard
Customers
Employees
Leads
Products
Deals
Interactions
Local CRM System
MySQL Database
Products
Manage your customer relationships.
Administrator
Products
Manage products, pricing and stock.
+ Add Product
ID Product Category Price Stock Actions
1 CRM Basic Software ₹15,000.00 100 Edit Delete
2 CRM Premium Software ₹30,000.00 50 Edit Delete
3 Business Analytics Service ₹20,000.00 30 Edit Delete
4 Marketing Automation Software ₹25,000.00 40 Edit Delete
5 Customer Support Suite Software ₹18,000.00 60 Edit Delete
6 Enterprise CRM Software ₹50,000.00 20 Edit Delete

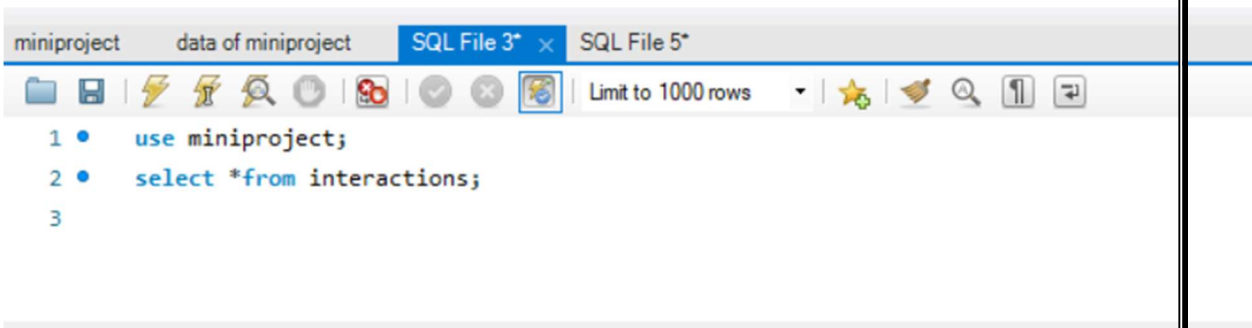

miniproject
data of miniproject
SQL File 3*
SQL File 5*
Limit to 1000 rows
use miniproject;
select * from customers;
select *from deals;
select *from employees;
select *from interactions;
Result Grid
Filter Rows:
Edit:
product_id product_name category stock
1 CRM Basic Software 100
2 CRM Premium Software 50
3 Business Analytics Service 30
4 Marketing Automation Software 40
5 Customer Support Suite Software 60
6 Enterprise CRM Software 20
NULL NULL NULL NULL

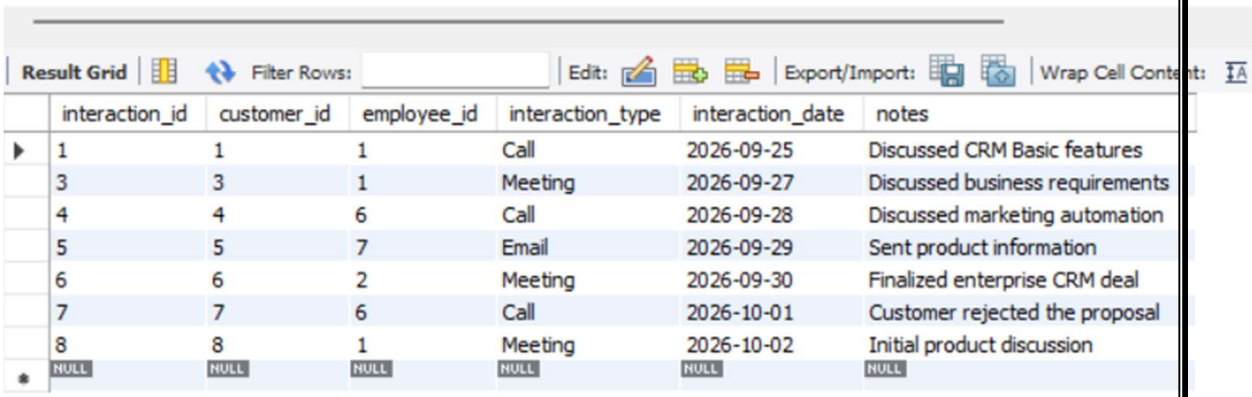




deal_id	customer_id	employee_id	product_id	deal_amount	stage	deal_date
1	1	1	1	15000.00	Closed Won	2026-09-25
3	3	1	3	20000.00	Proposal	2026-09-27
4	4	6	4	25000.00	Closed Won	2026-09-28
5	5	7	5	18000.00	Qualified	2026-09-29
6	6	2	6	50000.00	Closed Won	2026-09-30
7	7	6	2	30000.00	Closed Lost	2026-10-01
8	8	1	3	20000.00	Prospecting	2026-10-02
NULL	NULL	NULL	NULL	NULL	NULL	NULL







interaction_id	customer_id	employee_id	interaction_type	interaction_date	notes
1	1	1	Call	2026-09-25	Discussed CRM Basic features
3	3	1	Meeting	2026-09-27	Discussed business requirements
4	4	6	Call	2026-09-28	Discussed marketing automation
5	5	7	Email	2026-09-29	Sent product information
6	6	2	Meeting	2026-09-30	Finalized enterprise CRM deal
7	7	6	Call	2026-10-01	Customer rejected the proposal
8	8	1	Meeting	2026-10-02	Initial product discussion
NULL	NULL	NULL	NULL	NULL	NULL

- Outputs achieved
  The local Express server serves the frontend and exposes the customer API. The endpoint /api/customers returns customer data from MySQL, demonstrating that the frontend/backend/database connection works. Relational constraints were also observed during deletion: MySQL rejected removal of a customer referenced by a row in the Deals table. This confirms that referential integrity is active.
- Performance / Accuracy
  The prototype was verified through functional checks in a local development environment. No formal load, stress, or security testing was performed, so quantitative performance or accuracy claims are not made. For the current academic scale, API responses and database operations are suitable for demonstrating the intended workflow.
- Comparison with existing methods
  Compared with spreadsheet-based tracking, the database approach supports structured queries and referential integrity. Compared with commercial CRM products, this prototype is smaller and lacks advanced automation, reporting, multi-user permissions, and cloud deployment, but it provides direct learning experience in SQL schema design and API integration.

10.Conclusion and Future Scope

- Summary of achievements
  The CRM mini project demonstrates a complete basic connection between a web frontend, a Node.js/Express backend, and a MySQL database. It organizes CRM entities, retrieves customer records through an API, and relies on relational constraints to protect linked data. The project meets its educational goal of applying database design and SQL concepts in a working web application.
- Limitations and potential improvements
  Future versions can add login and role-based permissions, server-side validation, password hashing, prepared statements for every data operation, audit history, pagination, search/filter options, sales dashboards, and automated reports. The database connection should use environment variables, and the application should be deployed only after security testing, error handling, and backup procedures are implemented.

11. References

1. Oracle. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/
2. Node.js Documentation. https://nodejs.org/docs/latest/api/
3. Express.js Documentation. https://expressjs.com/
4. MDN Web Docs, JavaScript and HTTP. https://developer.mozilla.org/
5. MySQL2 package documentation. https://sidorares.github.io/node-mysql2/

12. Appendix

- Source Code Snippets
  Example Express route for retrieving customers:

```
app.get("/api/customers", (req, res) => {
  db.query("SELECT * FROM Customers", (err, results) => {
    if (err) return res.status(500).json({ error: "Database query failed" });
    res.json(results);
  });
});
```

  Use parameterized queries for user-supplied values. Do not expose database credentials in frontend files. This snippet illustrates the route concept; align it with the final server.js before submission.
- Additional screenshots
  Attach screenshots of the running application, API response, database tables, and successful/blocked operations. Verify that all screenshots correspond to the final version of the project.